

Datenbanken

Übung 8

Dozent: M. Sc. Burak Boyaci

Version: 19. Januar 2026

Wintersemester 25/26

1. Öffnen Sie pgAdmin 4 und verbinden Sie sich mit Ihrer Datenbank.
2. Schauen Sie sich bitte die Beispiele zu PL/pgSQL aus dem Skript noch einmal an und versuchen Sie die Beispiele nachzuvollziehen. Es ist sinnvoll, dass Sie die Beispiele selbst einmal in Ihrer Datenbank ausführen und die Ergebnisse nachvollziehen.
3. Schreiben Sie bitte eine gespeicherte Prozedur *Inflation* in PL/pgSQL, die folgendes tut: Bei Aufruf der Prozedur und Übergabe des Produktcodes als Parameter soll der Buyprice in der Products sich um 5% erhöhen. Außerdem soll auf der Konsole nach dem Aufruf der Prozedur folgende Information ausgegeben werden: 'Der Preis für das Produkt x hat sich um 5 Prozent erhöht.', wobei für x ein konkreter Produktcode stehen soll.

Rufen Sie bitte anschließend Ihre Prozedur durch Übergabe eines Productcodes auf.

4. Im Auftrag des Managements sollen Sie eine gespeicherte Funktion schreiben. Das Management braucht eine Funktion `get_order_count_by_status`, die den Status einer Bestellung in der Tabelle Orders abfragt und die Anzahl der Bestellungen mit diesem Status zurückgeben kann. Also: wenn die Funktion `get_order_count_by_status` aufgerufen und beispielsweise der Status shipped übergeben wird, soll die Anzahl der Bestellungen mit diesem Status ausgegeben werden. Die Anzahl soll als `total_order` ausgegeben werden.

Rufen Sie bitte anschließend Ihre Funktion durch Übergabe eines Status auf.

5. Erstellen Sie bitte eine Prozedur in PL/pgSQL, mit der der Einkaufspreis (buyprice) eines Produkts in der Tabelle Products um einen übergebenen Betrag erhöht wird. Die Prozedur soll zwei Parameter besitzen: den Produktcode (`v_productcode` vom Typ TEXT) und den Erhöhungsbetrag (`increase_amount` vom Typ INT). Aktualisieren Sie den buyprice nur für das Produkt, dessen productcode dem übergebenen Parameter entspricht.

Rufen Sie bitte anschließend Ihre Prozedur durch Übergabe des Productcodes 'S18_2581' und des konkreten Erhöhungsbetrag in Höhe von 10,99 auf (alter Buyprice: 49).

6. In der Tabelle Products wird für jedes Produkt jeweils ein erwarteter Umsatz (= msrp) und ein Kaufpreis (= buyprice) gespeichert. Das Management hat Sie als Entwickler beauftragt eine Funktion `get_profit` zu schreiben, die bei Übergabe des Productcodes als Parameter den MSRP, Buyprice und den erwarteten Profit zurückgibt. Es soll bei Aufruf der Funktion und Übergabe des Parameters Productcode folgendes zurückgegeben:

a) MSRP, Buyprice, Profit

b) Eine Information, die je nach Parameter, unterschiedlich ausfallen soll – wobei 3 Fälle unterschieden werden sollen:

1. Der Profit ist größer oder gleich 0: 'Ihre Eingaben: msrp = ... €, buyprice = ... €. Der Gewinn beträgt: ... €.'
2. Der Profit ist kleiner als 0: 'Ihre Eingaben: msrp = ... €, buyprice = ... €. Der Verlust beträgt: ... €.'
3. In allen anderen Fällen soll eine Fehlermeldung ausgegeben werden: 'Überprüfen Sie bitte Ihre Eingaben.'

Für die ... setzen Sie bitte jeweils die richtigen Parameter ein.

Rufen Sie bitte anschließend Ihre Funktion durch Übergabe des Productcodes 'S10_4757' auf.

Hinweis: Die nachfolgenden Aufgaben beziehen sich alle auf das Datenmodell von FloppyFlight das aus den drei Tabellen Players, Sessions und Events besteht.

7. Erstellen Sie bitte eine Funktion `get_average_score_by_group`, die als Eingabeparameter eine Gruppennummer (`p_group` vom Datentyp INT) erhält. Die Funktion soll den durchschnittlichen `final_score` aller Sessions zurückgeben, deren Spieler der angegebenen Gruppe angehören. Beachten Sie, dass die Spalte `group` in der Tabelle `players` ein reserviertes Schlüsselwort ist und deshalb als "group" maskiert werden muss.
Die Funktion soll als Rückgabewert ein numerisches Ergebnis liefern (numeric) und den Durchschnitt über alle gefundenen Sessions bilden.
8. Erstellen Sie bitte eine Prozedur `rename_player` (`p_player_id` text, `p_new_name` text), die den Namen eines Spielers in `players.player_name` aktualisiert.
Geben Sie nach erfolgreichem Update eine Meldung per RAISE INFO aus: „Spieler heißt nun “). Falls kein Spieler gefunden wird, geben Sie eine passende Info aus.
9. Schreiben Sie bitte eine Prozedur `reset_session_score`(`p_session_id` text), die den `final_score` der in `sessions` gespeicherten Session mit der aufgerufenen `session_id` auf 0 setzt. Verwenden Sie dabei IF FOUND THEN, um anschließend per RAISE INFO auszugeben,
 - „Score zurückgesetzt“, wenn eine Zeile aktualisiert wurde,
 - „Nicht gefunden“, wenn keine entsprechende Session existiert.

Führen Sie danach einen Aufruf der Prozedur beispielhaft mit folgender Session-ID durch: '0693da1d-d8f2-4c8b-9daa-861b5b5911a2'

10. Schreiben Sie bitte eine Prozedur `double_final_score`, die bei Aufruf den `final_score` in der Tabelle `Sessions` verdoppelt. Es soll außerdem die Anzahl der verdoppelten Einträge auf der Konsole als Info ausgegeben werden: 'Die Anzahl der verdoppelten Einträge beträgt: '.